

TTP: Time-Triggered Protocol for Fault-Tolerant Real-Time Systems: A Technical Review, Contextualization, and Critical Evaluation

[Jaleel Ur Rehman]

Electronics Engineering

[Hochschule Hamm-Lippstadt]

jaleel.ur-rehman@stud.hshl.de

Abstract: The Time-Triggered Protocol (TTP), developed by Kopetz and Grünsteidl in 1993, is an integrated communication protocol that supports real-time control systems requiring fault tolerance. The protocol offers predictable message delivery, clock synchronization, membership service, and redundancy handling in one integrated protocol. In this paper, we describe the technology behind the protocol, provide a context to the protocol within real-time communication protocols, and critically review the protocol assumptions, strengths, and weaknesses. In particular, we examine the TTP time-division multiple access (TDMA) scheduling algorithm, C-state agreement protocol, fail-silently nodes, and four fault-tolerant system configuration modes. Furthermore, we compare TTP with event-based protocols like CAN and token-based protocols. We also present its application domain with modern applications including automotive electronics. Finally, we suggest some open problems for contemporary real-time systems engineering.

Index Terms: time-triggered protocol, real-time systems, fault tolerance, TDMA, clock synchronization, embedded systems, distributed systems, automotive electronics

I. INTRODUCTION

Distributed real-time control systems face two fundamental engineering constraints: timely performance and dependable behavior. Timely behavior implies the satisfaction of hard temporal requirements on computations and communications; dependable behavior involves continuing operation correctly even in the presence of faults in hardware, failures in communications, or other disturbances. Satisfying both requirements concurrently can be challenging when active fault tolerance based on hardware and software redundancy is required. Real-time communication protocols flourished in the early 1990s, which can be classified into two categories: event-triggered architecture, in which actions are triggered based on changes in state, and time-triggered architecture, where the entire process is based on global clock [1]. Time Triggered Protocol (TTP), proposed by Kopetz and Grünsteidl [1], provides a

comprehensive solution by statically allocating communication bandwidth, globally synchronizing clock and including all services needed for a fault tolerant system such as predictable message transfer, membership services, group acknowledgments, clock synchronization, rapid change modes and redundancy management in one protocol without introducing any additional messages. The seminar paper shall be divided into the following parts. Section II gives a technical explanation of the fundamental operations of the TTP algorithm. Section III situates the TTP within the current research environment and compares it with other protocols. Section IV evaluates the assumptions, strengths, and weaknesses of TTP. Section V deals with the areas where TTP can be implemented.

II. TECHNICAL DESCRIPTION OF TTP

A. Architectural Assumptions

TTP is targeted at the distributed system comprising a set of fail-silent nodes that communicate using two replicas of broadcast communication channels [1]. Every node has its own communication controller, having two duplex communication ports. To support node fault tolerance, nodes can be replicated and organized into Fault Tolerant Units (FTUs), where the condition of availability of any single node implies the availability of the entire FTU. The communication channel is represented as a passive broadcast LAN channel, as a broadcast bus able to transport only one message at a time. The access to the channel is carried out via the Time Division Multiple Access (TDMA) scheme, which is determined by globally synchronized time base.

B. Failure Model

TTP assumes fail-silent nodes and omission-failure channels. The assumed failure rates for the four failure types are summarized in Table I [1].

The assumption of fail-silent nodes is fundamental to TTP's design. A node either delivers correct results or produces no output at all. This abstraction is enforced through space and time redundancy at the hardware level, combined with error

TABLE I
ASSUMED FAILURE RATES IN TTP

Failure Type	Rate (h^{-1})
Permanent Node Failure	10^{-3}
Permanent Channel Failure	10^{-4}
Transient Node Failure	10^{-5}
Transient Channel Failure	10^{-6}

detection mechanisms that shut down the node before any erroneous output is transmitted [1].

C. System Configuration Classes

TTP defines four fault-tolerant system configuration classes with increasing levels of redundancy, as shown in Table II [1].

TABLE II
TTP FAULT-TOLERANT CONFIGURATION CLASSES

Tolerance of	I	II	III	IV
Perm. Node Fail.	0	1	1	2
Perm. Comm. Fail.	1	1	1	1
Trans. Node Fail.	0	1/Rcint	1/Rcint	1/TR
Trans. Comm. Fail.	1 of 2	1 of 2	3 of 4	3 of 4

Class I represents the simplest configuration with one node per FTU and two frames per FTU, suitable when cabling failures are dominant. Class IV, the most robust configuration, uses two active nodes and one shadow node per FTU with four frames per FTU, and can tolerate up to two permanent node failures.

D. Sparse Time Base and TDMA Scheduling

The idea of using the sparse time base is one of the key features used in the development of TTP. Instead of communicating randomly in continuous time, in TTP, all send and receive operations are synchronized in accordance with a lattice grid [1]. The size of this grid is defined by such factors as clock synchronization accuracy, quartz precision, and channel characteristics. It is called basic time granule g . Given the 250 kbit/s channel, the length of one granule equals approximately 2 bits. This feature drastically decreases the number of scenarios required for modeling and analyzing of the system. Access to bus is completely regulated by the global time in TTP. One FTU slot includes 1 or 2 frame slots according to fault tolerance classes. TDMA pattern, messages, and task schedule are fixed for every operational mode of the system.

E. Controller State (C-State)

Each TTP communication controller's state, the C-state, contains three pieces of information: a mode field, a time field, and a membership field [1].

- **Mode field:** Identifies the current operational mode. A TDMA pattern, message format definitions and dispatcher table have been defined for each mode.
- **Time field:** Indicates the global internal time and where in the cyclic mode the control lies.

- **Membership field:** The membership of a set of active or inactive FTUs is represented by a bit vector.

Instead of sending the C-state in each transmitted frame, TTP makes use of a smart CRC calculation involving not only the data but also the concatenated C-state. If a CRC check fails on a received frame, it means that either the data are corrupted or sender/receiver have different C-states, in both situations the frame will be rejected. This way, a valid frame is only accepted when both sender and receiver have an agreed upon C-state, the same mode, time, and membership.

F. Clock Synchronization

TTP enables fault tolerance in terms of internal clock synchronization without overheads in terms of frame size and number of frames. Given that all receivers know in advance when the frame is going to be sent, the difference between the expected time of reception and the actual one gives an estimate of the difference in clocks between the sender and the receiver. Periodic applications of the fault-tolerance averaging algorithm are preferred to achieve global clock coherence [4]. External clock synchronization is outside the scope of TTP; it may be achieved by giving the node access to an external time reference.

G. Membership Service

The membership service offered by TTP offers a reliable way of knowing whether an FTU is active or inactive. Whenever an FTU sends a frame, it sets its membership bit to one. The receiver, on the other hand, modifies the membership bit of the sending FTU accordingly. If a correct frame is not received from an FTU in its turn to send, its membership bit is set to zero. There exists a method in TTP for preventing the creation of cliques, disjoint sets of nodes with different opinions of the rest of the set. When a receiver receives fewer frames than it discards in one round of TDMA, it becomes evident that its C-state does not match the majority of the rest of the network.

H. Mode Change

TTP Mode Changes facilitate the system changing context (for example, startup, operation, and emergency modes). A mode change is indicated when the next mode number is entered into the mode change subfield in the control field of the next membership cycle. At most, the delay in performing a mode change will be up to one TDMA cycle, but possibly less if the requesting node receives a special slot for the mode change notification.

I. Redundancy Management and Initialization

Redundancy management includes the process of removing faulty nodes as well as adding back repaired nodes into the network. Re-introduced nodes undergo a standardized start-up sequence, which includes carrying out a self-diagnostic test, resetting the local clock, listening on the bus for a prescribed time period I_1 , and then deciding whether to carry out a cold start because of absence of traffic or to join an active

ensemble because of receipt of an I-frame. The initialization frame (I-frame) carries the C-state of the transmitting node in its payload area. Shadow nodes in Class IV systems are immediately available as warm backups because they step into the sending slot of a working node if that slot has been identified as being unacknowledged.

III. SCIENTIFIC CONTEXTUALIZATION

A. Event-Triggered vs. Time-Triggered Architectures

The essential difference between the two architectural models is the event triggering technique [1], [2]. ET-based systems such as CAN [3] and J1850 employ a demand-driven scheduling policy where bandwidth is assigned dynamically according to the state changes. They are efficient in terms of low load utilization but suffer from load-dependent worst-case timing, complex schedulability analysis, and priority inversion during fault handling. In contrast, TT systems utilize a static bandwidth assignment in a TDMA schedule. All processes and communication procedures are periodic events that are triggered by external global time. As a result, load-independent worst-case timing, simple schedulability analysis, and determinism of replicas are ensured [1].

B. Comparison with CAN

CAN employs a contention-based mechanism for arbitration [3]. Although it is suitable for applications under light load and not critical to safety, CAN's maximum latency depends on the load, and it cannot be easily bounded when faults occur without using an additional protocol layer. TTP's 53.3% data efficiency is significantly higher than competing protocols, achieved by eliminating the need for sender addresses, message names, and explicit acknowledgment messages through a priori knowledge and the C-state agreement mechanism [1].

C. FlexRay

FlexRay was a leading safety-critical automotive standard during the 2000s, offering a hybrid static/dynamic segment capable of supporting both TT and ET communications [6]. In terms of its architecture, FlexRay presents a compromise solution between the TT model of TTP and ET protocols, acknowledging the limitations of the latter.

IV. CRITICAL EVALUATION

A. Strengths

Temporal encapsulation and testability. A major benefit of the TTP protocol is temporal encapsulation of nodes, which means that all node interactions occur within a predefined time period. It allows developing and testing each node independently from others in terms of the value and time domains [1]. In contrast, ET architectures suffer from global effects on the entire system due to timing modifications in a local task; thus, after each modification, it is necessary to perform system-wide regression tests. With TTP, such dependencies are eliminated and testing becomes significantly easier.

Replica determinism. Replica determinism is guaranteed in the TTP protocol since replicated nodes make their state

changes simultaneously in the time domain and hence perform the same sequence of operations, which leads to identical results in replicas. Thus, it provides prerequisites for transparent active redundancy [1]. ET architectures offer no equivalent mechanism.

Integrated services. The TTP protocol provides clock synchronization, node membership determination, message acknowledgement, and redundancy management mechanisms without increasing the number of messages and overhead frames sent in the network [1].

High data efficiency. TTP achieves 53.3% data efficiency compared to 24–27% for competing protocols, by exploiting a priori knowledge to eliminate message names and acknowledgment messages [1].

B. Limitations and Risks

Inflexibility due to fixed scheduling. The most significant weakness of TTP is the fixed TDMA schedule that it uses. Adding a new FTU node or changing a message format in the system necessitates creating a new mode definition and reconfiguring the schedule for all nodes in the ensemble.

Inefficiency of bandwidth usage when load is small. In TTP, communication bandwidth is statically allocated regardless of actual demand. When many nodes do not have any messages to send in a given TDMA cycle, their allocated bandwidth remains unused. In protocols such as CAN that are triggered by events, this excess can be put into efficient use [3].

Fail-Silent Assumption. The accuracy of TTP is highly dependent on the fail-silent property of nodes. For making fail-silent nodes, hardware redundancy (in space or time) and error detection mechanisms that can detect all internal errors prior to producing any incorrect output are required. This is difficult to do in real life and increases implementation costs. Byzantine faults, which happen when a node transmits inconsistent values to multiple receivers, cannot be handled using the failure model used by TTP [5].

Clock Synchronization Scalability. The clock synchronization algorithm of TTP can handle clock synchronization for small to moderately sized ensembles [4]. However, when there are large numbers of nodes distributed across great distances, clock synchronization becomes increasingly inaccurate.

C. Assumptions That May Be Unrealistic

Periodic workloads only. The assumptions behind TTP include the presumption that periodic communication constitutes the majority of messages to be communicated over the network. Sporadic event-based messaging can be accommodated by means of mode changes, but the protocol is not well-suited to use cases involving predominantly aperiodic communication traffic [2]. Autonomous driving applications provide a modern example, with highly variable amounts of sensor data produced as a function of environmental complexity.

Stable system topology. TTP assumes a static system topology known beforehand [1]. Dynamically adding and removing nodes in the network is possible, but would require

a complete redeployment of schedules. Consequently, this protocol is unsuited for open systems or networks requiring plug-and-play capability.

Assumptions about fault occurrence. The operation of both the membership and mode change protocols depend on the assumption that no node fails at a specific instant when a change of configuration takes place. Under an asynchronous faults model, this assumption can never be guaranteed to hold [5].

V. APPLICATION DISCUSSION

TTP or its derived versions are suitable for systems having the following properties:

- **Real-time safety-critical control with time guarantees:** Automotive chassis control systems (brake-by-wire and steer-by-wire), aircraft flight control systems, industrial robotics control, and railway signalling systems demand guaranteed timing and dependability that TTP provides [1].
- **Predictable periodic data transfer:** Engine control systems where there is periodic sampling of data (temperature, pressure, and positions) as well as periodic control command transfers fit well within the TTP TDMA model [1].

On the other hand, TTP is not suitable for in-car entertainment systems, over-the-air firmware updates, or camera/LiDAR sensor feeds whose bandwidth is too inconsistent to allow static scheduling. Similarly, industrial IoT networks where sensors and actuators are regularly added or deleted would require frequent schedule redeployment, and low-cost body electronics such as window and seat controls benefit more from the simplicity of CAN [3].

VI. CONCLUSION

In the realm of designing fault-tolerant real-time systems, the Time-Triggered Protocol stands out as an important breakthrough. The incorporation of clock synchronization, membership services, redundancy handling, and assured delivery of messages in one efficient protocol, along with using a priori information about message content to eliminate redundant fields, allows TTP to achieve levels of data efficiency and time predictability that event-triggered protocols cannot offer [1].

Time predictability, temporal encapsulation of nodes, and replica determinism provide solutions to fundamental problems in designing safe distributed control systems. These features of TTP have influenced the design of subsequent protocols such as FlexRay [6], and the basic tenets of the TTP approach have found their way into modern Time-Sensitive Networking standards [7].

However, the lack of flexibility, the need to maintain a static schedule, the assumption of fail-silent nodes, and a general lack of scalability to dynamic or non-periodic workloads are limitations that should not be overlooked when implementing a solution based on TTP [2]. For applications with periodic workloads in safety-critical domains, TTP and its derivatives are appropriate. Other systems may find better solutions.

TTP's key principle of using global synchronization of a sparse time base to achieve correct, efficient, and analyzable distributed computation is still highly relevant to today's cyber-physical systems design, real-time operating systems, and safety-critical embedded systems development [1], [7].

REFERENCES

- [1] H. Kopetz and G. Grünsteidl, "TTP: A Time-Triggered Protocol for Fault-Tolerant Real-Time Systems," in *Proc. 23rd Int. Symp. on Fault-Tolerant Computing (FTCS-23)*, pp. 524–533, Toulouse, France, June 1993.
- [2] H. Kopetz, "Event-Triggered versus Time-Triggered Real-Time Systems," in A. Karshmer and J. Nehmer, editors, *Proc. Int. Workshop on Operating Systems of the 90s and Beyond*, Lecture Notes in Computer Science, vol. 563, pp. 87–101, Springer-Verlag, 1991.
- [3] *SAE Handbook, Vol. 2*. Society of Automotive Engineers, 400 Commonwealth Drive, PA, USA, 1992.
- [4] H. Kopetz and W. Ochsenreiter, "Clock Synchronization in Distributed Real-Time Systems," *IEEE Trans. on Computers*, vol. 36, no. 8, pp. 933–940, Aug. 1987.
- [5] L. Lamport, R. Shostak, and M. Pease, "The Byzantine Generals Problem," *ACM Trans. on Programming Languages and Systems*, vol. 4, no. 3, pp. 382–401, 1982.
- [6] FlexRay Consortium, *FlexRay Communications System: Protocol Specification, Version 2.1*, 2005.
- [7] IEEE Standards Association, "IEEE 802.1Qbv: Enhancements for Scheduled Traffic," *IEEE Standard*, 2015.